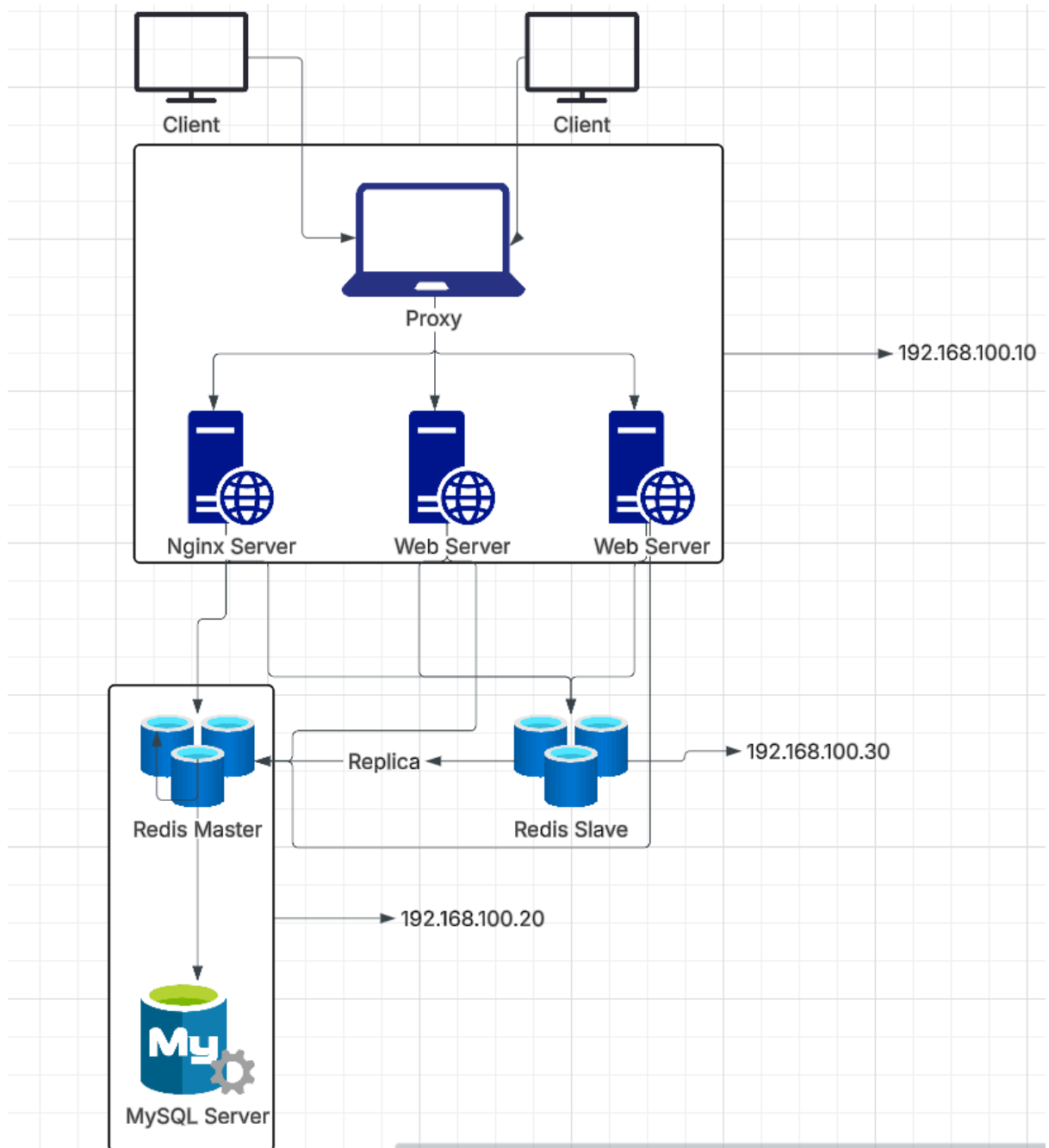


Redis

I. Mô hình



II. Kết quả

TH1: Dùng Redis làm cache cho Web/App

B1: Trên máy WebServer (192.168.100.10)

Tạo 1 web app như sau :

```
from flask import Flask, jsonify
import redis
import pymysql
import json
import time

app = Flask(__name__)

# Redis Master
r = redis.Redis(
    host='192.168.100.20',
    port=6379,
    password='vannquyen22222222',
    decode_responses=True
)

# MySQL
db_config = {
    "host": "192.168.100.20", # sủa lại nếu MySQL của bạn ở IP khác
    "user": "vannquyen",
    "password": "AppPass123!",
    "database": "labdb",
    "cursorclass": pymysql.cursors.DictCursor
}

@app.route("/")
def home():
    return "Redis Cache Demo is running!"

@app.route("/products")
def get_products():
    cache_key = 'products:all'
    start = time.time()
    # 1. Thử lấy từ Redis trước
    cached_data = r.get(cache_key)
    if cached_data:
        duration = round((time.time() - start) * 1000, 2)
        return jsonify({
            "source": "redis_cache",
            "duration_ms": duration,
            "data": json.loads(cached_data)
        })
    # 2. Nếu Redis chưa có, lấy từ MySQL
    connection = pymysql.connect(**db_config)
    with connection:
        with connection.cursor() as cursor:
            cursor.execute('SELECT * FROM products')
            products = cursor.fetchall()
    # 3. Lưu vào MySQL, chờ 2 giây để bạn có thể thấy khác biệt
    time.sleep(2)
    # 4. Lưu kết quả vào Redis trong 60 giây
    r.setex(cache_key, 60, json.dumps(products))
    duration = round((time.time() - start) * 1000, 2)
    return jsonify({
        "source": "mysql",
        "duration_ms": duration,
        "data": products
    })

@app.route("/clear-cache")
def clear_cache():
    r.delete('products:all')
    return "Cache cleared"

if __name__ == "__main__":
    app.run(host='0.0.0.0', port=8080)
```

B2: Trên máy MySQL Server (192.168.100.20), em khởi tạo Database như sau :

```
CREATE DATABASE labdb;

USE labdb;

CREATE TABLE products (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100),
    price INT
);

INSERT INTO products (name, price) VALUES
('Laptop Dell', 15000000),
('Keyboard Logitech', 500000),
('Mouse Razer', 700000);

CREATE USER 'appuser'@'192.168.100.10' IDENTIFIED BY 'AppPass123!';
GRANT ALL PRIVILEGES ON labdb.* TO 'appuser'@'192.168.100.10';
FLUSH PRIVILEGES;
```

B3: Trên máy Web Server (192.168.100.10), em chạy Web

B4: Em dùng máy Nginx (192.168.100.10, ssh 1 terminal khác), tiến hành lấy Database 2 lần, để kiểm tra lần 1 Web có lấy Database từ MySQL hay không, và lần 2 lấy Database từ Redis Cache. Kết quả chính xác như hình bên dưới

```
vannquyen@nginx:~$ curl http://192.168.100.10:8080/products
{"data":[{"id":1,"name":"Laptop Dell","price":15000000}, {"id":2,"name":"Keyboard Logitech","price":500000}, {"id":3,"name":"Mouse Razer","price":700000}], "duration_ms":2083.39, "source":"mysql"}
vannquyen@nginx:~$ curl http://192.168.100.10:8080/products
{"data":[{"id":1,"name":"Laptop Dell","price":15000000}, {"id":2,"name":"Keyboard Logitech","price":500000}, {"id":3,"name":"Mouse Razer","price":700000}], "duration_ms":1.52, "source":"redis_cache"}
vannquyen@nginx:~$
```

TH2: Kiểm tra Redis Master → Replica

Mục tiêu của test này: ghi dữ liệu vào **Redis Master**, sau đó kiểm tra xem **Redis Replica** có nhận được dữ liệu không.

B1: Trên máy Redis Master :

- ```
- redis-cli -a 'Vanquyen2522005@'
- SET user:1 "Pham Van Quyen"
```

- GET user:1 -> Kết quả trả về : Pham Van Quyen

## B2: Trên máy Redis Slave

Em kiểm tra xem có đọc được dữ liệu trên Master không và kết quả thành công: “

```
vanquyen@Redis-Replication:~$ redis-cli -a 'Vanquyen2522005@'
Warning: Using a password with '-a' or '-u' option on the command line interface
may not be safe.
127.0.0.1:6379> GET user:1
"Pham Van Quyen"
```

```
Vanquyen@Redis-Replication:~$ redis-cli -h Vanquyen2522005@ INFO replication
Warning: Using a password with '-a' or '-u' option on the command line interface may not be safe.
Replication
role:slave
master_host:192.168.100.20
master_port:6379
master_link_status:up
master_last_io_seconds_ago:1
master_sync_in_progress:0
slave_read_repl_offset:3468
slave_repl_offset:3468
slave_priority:100
slave_read_only:1
replica_annotated:1
connected_slaves:0
master_failover_state:no-failover
master_replid:4acd763c7821348447a63be7b7fd879cb1fe6503
master_replid2:00
master_repl_offset:3468
second_repl_offset:-1
repl_backlog_active:1
repl_backlog_size:1048576
repl_backlog_first_byte_offset:15
repl_backlog_histlen:3454
```

- Trạng thái của Redis Slave

Ngoài ra, Em test curl từ 1 máy cùng dải mạng LAN 192.168.100.0 (Đóng vai trò là Client)

```
vanquyen@mail:~$ curl -k -I https://192.168.100.10
HTTP/2 200
server: nginx
date: Mon, 08 Jun 2026 05:24:16 GMT
content-type: text/html; charset=utf-8
vary: Accept-Encoding
set-cookie: MCSESSID=4cb2504368346072264c9e38632e9ccd; path=/; secure; HttpOnly; SameSite=Lax
expires: Thu, 19 Nov 1981 08:52:00 GMT
cache-control: no-store, no-cache, must-revalidate
pragma: no-cache
strict-transport-security: max-age=15768000;
x-content-type-options: nosniff
x-robots-tag: none
x-download-options: noopen
x-frame-options: SAMEORIGIN
x-permitted-cross-domain-policies: none
referrer-policy: strict-origin

vanquyen@mail:~$
```

